



# AGV

**Software Task Round**  
**Autonomous Ground Vehicle**

## **TASK 1 - Aw Shii, Here we flow again...**

Seeing Motion Like Humans Do

Think of merging onto a highway at dusk: your brain instinctively tracks the speed of approaching cars using subtle visual cues, not raw speedometers. Optical flow replicates this biological motion sense for machines, solving critical problems other sensors miss.

### **SUBTASK 1: IMPLEMENTING OPTICAL FLOW** **CJ NEEDS HELP!**

You are expected to implement the Lucas-Kanade sparse optical flow estimation algorithm specified in this article on clips of your choice from this video. You are allowed to use basic OpenCV functions: `cv2.imread`, `cv2.imshow`, etc., and you may additionally use `cv2.goodFeaturesToTrack`. You may refer OpenCV documentation.

Solution:

Aim is to track the moving object using the concept of optical flow and Lucas-Kanade sparse optical flow estimation algorithm and also the coarse to fine flow

estimation (simply say concept of pyramids) .

will be trying to apply this concept on the clip between 15-20sec in the provided video

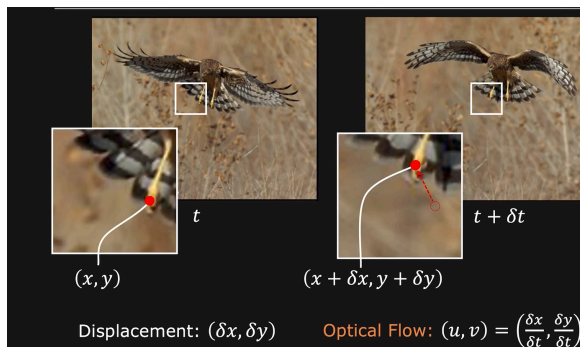
taken care by :

```
START_FRAME = 900    # 15 seconds × 60 fps
END_FRAME    = 1200   # 20 seconds × 60 fps
```

got the basic code from the official opencv documentation

## theory :

optical flow basically works by detecting change in brightness patterns between 2 consecutive frames . but it will be difficult so we made some assumptions :



### Assumption #1:

Brightness of image point remains constant over time

$$I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t)$$

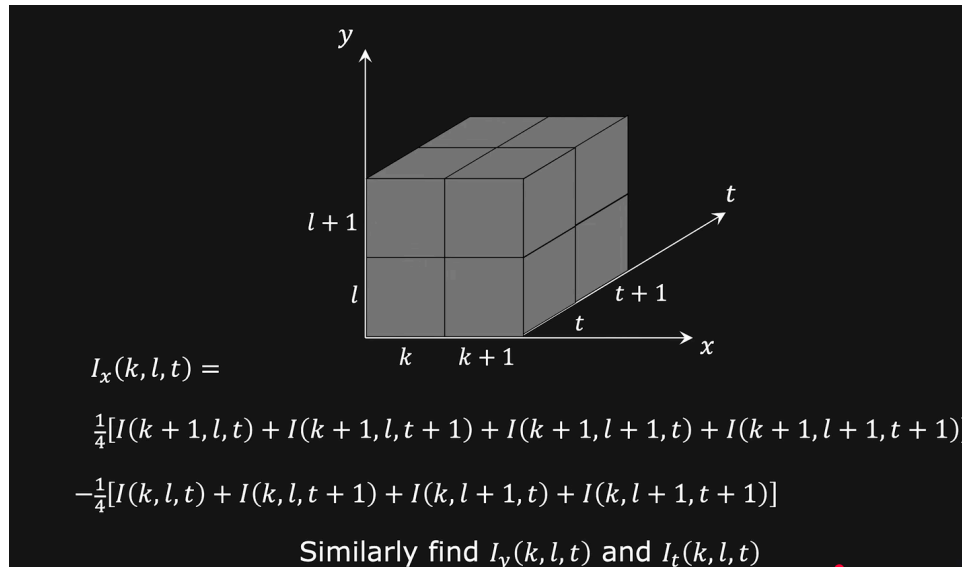
### Assumption #2:

Displacement  $(\delta x, \delta y)$  and time step  $\delta t$  are small

therefore taking these assumptions into account and solving we get :

$I_x u + I_y v + I_t = 0$  ,  $(u, v)$  : optical flow

now we can easily get  $(I_x, I_y, I_t)$  from two frames



taken care by this function in code `compute_gradients(frame1, frame2)`

### Lucas-Kanade sparse optical flow estimation algorithm :

assumption : for each pixel , assume motion field , and hence optical flow  $(u,v)$  , is constant within a small neighborhood  $W$ .

in  $W$  their are many points such as  $(k,l)$  which satisfies

$$I_x(k,l)u + I_y(k,l)v + I_t(k,l) = 0 \text{ and if size of } W \text{ is } n \times n$$

therefore

In matrix form:

$$\begin{bmatrix} I_x(1,1) & I_y(1,1) \\ I_x(k,l) & I_y(k,l) \\ \vdots & \vdots \\ I_x(n,n) & I_y(n,n) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} -I_t(1,1) \\ -I_t(k,l) \\ \vdots \\ -I_t(n,n) \end{bmatrix}$$

\*ATA should be invertible matrix

### Least Squares Solution

Solve linear system:  $Au = B$   
 $A^T A u = A^T B$  (Least-Squares using Pseudo-Inverse)

In matrix form:

$$\begin{bmatrix} \sum_W I_x I_x & \sum_W I_x I_y \\ \sum_W I_x I_y & \sum_W I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} -\sum_W I_x I_t \\ -\sum_W I_y I_t \end{bmatrix}$$

$A^T A$  (Known)  $2 \times 2$        $u$  (Unknown)  $2 \times 1$        $A^T B$  (Known)  $2 \times 1$

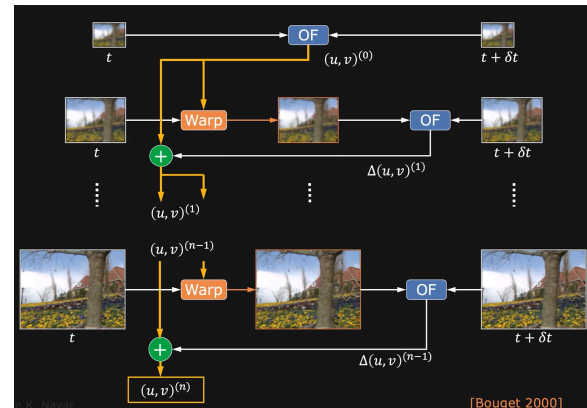
Indices  $(k, l)$  not written for simplicity

$$u = (A^T A)^{-1} A^T B$$

Fast and Easy to Solve

`lk_optical_flow_point(Ix, Iy, It, point, win_size=15)` function for this part

but there is one problem , these calculations and formulas based on the assumption that change in x and y is small for small t . this method will fail for fast moving objects , to tackle this we introduced the coarse to fine flow estimation(concept of pyramids)



The image pyramid concept involves creating a multi-scale representation of an image by repeatedly smoothing it with a Gaussian filter and then downsampling it (typically by a factor of two). This process generates a stack of images, where the base is the original full-resolution frame and each subsequent higher level is half the width and height of the one below it. In the context of Lucas-Kanade, the algorithm first calculates motion at the coarsest (smallest) level where large pixel displacements appear small; it then propagates this initial flow estimate down to the finer, higher-resolution levels as a starting "guess" to be incrementally refined.

now main part point selection , which is done using shi-tomasi parameters ,they work on the principle that matrix ATA must be well conditioned and invertible . a matrix is said to be well conditioned , say the matrix ATA has two eigen values say lemnda1 and lemnda2 , if below conditions satisfy

lemnda1 > e and lemnda2 > e (lemnda1 and lemnda2 must not be near or equal to 0) & lemnda1 greater than or equal to lemnda2 , but not lemnda1 >>>> lemnda2

```
# Shi-Tomasi params
feature_params = dict(maxCorners=100,
                      qualityLevel=0.1,
```

```
minDistance=7,  
blockSize=7)
```

finally varying these parameters helped in tuning with the frame range on which we are implementing optical flow.

## Implementation in code :

- `compute_gradients(frame1, frame2)` : Computes the spatial ( `Ix` , `Iy` ) and temporal ( `It` ) derivatives required for the Lucas-Kanade equation. These gradients are averaged over a 2×2 neighbourhood
- `lk_optical_flow_point(Ix, Iy, It, point, win_size=15)` : Solves the Lucas-Kanade equation for **a single point** using a local window (patch). It computes the optical flow vector (u, v) = movement of that point.
- `track_points(frame1, frame2, points, win_size=15)` : Tracks **multiple points** from `frame1` to `frame2` using the single-point LK function for each point.
- `build_pyramid(image, max_level=2)` : Builds a Gaussian image pyramid **without using OpenCV's** `pyrDown` – implements blurring and subsampling manually.
- `pyramidal_lk(frame1, frame2, points, win_size=15, max_level=2)` : Implements **pyramidal Lucas-Kanade** – the full multi-scale version that tracks points from coarse to fine levels, providing larger motion range and better accuracy.
- **Main loop** ( `if __name__ == "__main__"` ) : Demonstrates the custom pyramidal LK on a video file. Tracks Shi-Tomasi corners over time, draws trails, and refreshes points periodically.

here is the final code for subtask 1 .

```
subtask1.py
```

## SUBTASK 2: GOING WITH THE FLOW

Let us now dive into a very interesting application of optical flow methods: moving your ego (a robot or an autonomous car) towards a target while avoiding obstacles.

Your task here is to use the Lucas-Kanade algorithm in conjunction with a few methods in order to implement a make-shift controller that reaches a given goal in a cool simulator.

Before proceeding with the task, you are required to pull and set up the docker container as mentioned in the compulsory task. Operate only from the home directory.

Launch the container and do a pip install of pybullet ,then create a python file where you will be coding up most or all of the following task.

Objective: Use sparse optical flow (Lucas-Kanade method) to navigate to the end of the racetrack avoiding the obstacles in the provided setup (using the given script).

You must follow these steps (refer to the paper linked below for more clarity):

- Set up a camera feed and capture RGB frames from Pybullet's agent camera.
- Compute optical flow between consecutive frames to identify motion vectors.
- Detect Focus of Expansion (FOE) to infer agent motion direction and distinguish ego-motion from obstacle motion.
- Generate a visual potential field where obstacles create repulsive vectors and the target (e.g., a doorway) creates an attractive vector.
- Integrate with Pybullet's API to steer the agent using gradient based control.

You must go through this this paper and understand its math (at least at a high-level) to accomplish this. Apart from your implementation of Lucas-Kanade

method, you may use any library function from OpenCV and NumPy. Go through their documentation.

Use as many resources as you want (including LLMs) for this subtask.

### SOLUTION:

initially i thought this task would be easy ,

initially the car was going in unpridictable directions , sometimes perpendicularly hitting the obstacle or sometimes just after simulation starts it takes sharp turn and then out of the track .

than tried adding different functions like realignment , dont react early for far away objects , ignore road strips(basically ignore clusters ) , etc but max it was able to cross first object than either out of the track or staright into second object .

changed the approach , removed every function and added new concept , react only when you see any irregularity in brightness , and react opposite to the direction where irregularity was found . it somehow worked upto 3 obstacles with realignemnet function and parameter tuning .

than freshly restarted again . with functions given in research paper .

changing these parameters one by one and applying bisection method , helped in tuning .

## **Working Phases :**

goal : stay on road , avoid obstacles , reach blue wall at the end

### **phase 1 : capturing the world :**

camera capture , lane detection , goal tracking

### **phase 2 : optical flow**

find corners ; track movement ; calculate trajectory(foe) , depth(ttc)

### **phase 3 : force**

Repulsion , Attraction , the resultant vector

### **phase 4 : control and recovery**

steering and speed , realignment

### **phase 5 :**

green/red dots , yellow repulsion arrows , net field pink arrow .

## **functions used :**

- `get_agent_camera`
- `get_road_center` : finds the leftmost and rightmost road pixels, and returns the midpoint. Used for lane keeping.
- `calculate_foe` : Computes the **Focus of Expansion (FOE)** – the point in the image from which all optical flow vectors appear to emanate , used later to estimate time-to-contact.
- `calculate_ttc` : **Time-To-Contact for each tracked feature point.  $TTC = \text{distance from the point to the FOE} / \text{magnitude of its optical flow}$ . Lower TTC means the object is closer and approaching faster.**
- `filter_dense_clusters` : removes groups of points that are too densely packed in a small image patch (e.g., road markings, texture on the road surface). These are not real obstacles and would otherwise cause false avoidance behaviour.
- `find_goal_centroid` : Detects the blue “goal wall” by thresholding the HSV image for blue colours. Returns the centroid (x, y) of the largest blue blob.
- `compute_steering_and_speed` : The core **vision-only control** function. It calculates: Obstacle avoidance, Road centering , Goal attraction
- `main` : initialises the simulation, captures frames, runs optical flow, detects road/obstacles/goal, computes steering & speed, applies controls, and displays visualisation. Also includes a recovery state after sharp avoidance manoeuvres.

using above functions it actually worked very nicely :

`agv2.py`

- `compute_potential_field` : Builds a 2D visual potential field in image coordinates:

**Repulsive forces** from obstacles: for each tracked feature with TTC below threshold, a vector points *from* the obstacle *toward* the ego centre (pushing the car away). Magnitude increases as TTC decreases.

**Attractive force** toward the goal (blue wall centroid or last known x-position).



agvsb2.py

agv2.py performs better than agvsb2.py but agv2.py lacks `compute_potential_field`

## Problems faced :

1. noise due to white strips on road , results in unpredictable turns
2. detecting far away objects due to which car oscillates and crashes into obstacle
3. very less difference even after changing below parameters significantly , therefore tuning took too much time .

```
def compute_steering_and_speed(  
    foe, good_old, ttc, goal_point, last_known_goal_x,  
    road_center_error,      # normalized road offset from image centre  
    # Gains (tunable)  
    k_obstacle=25.0,        # repulsion strength  
    k_road=0.0,             # road centering gain  
    k_goal=0.8,             # goal attraction  
    ttc_threshold=17.5,     # only react to obstacles with TTC < this  
    max_steer=0.7           # max steering angle (radians)  
):
```

\*\*\*one edit i did in `simulation_setup.py` was to remove centre white strips of road as it was creating noise

by Somil Goyal

25EC10076